CodeChef.java

```
import java.io.*;
import java.util.*;
public class Codechef {
    public static void solve(MyScanner sc){
        int n=sc.nextInt();
        // print("----------------------");
    }
    public static void main(String[] args) {
        MyScanner sc = new MyScanner();
        int test = sc.nextInt();
        while (test-->0) {
            solve(sc);
        }
        out.flush();
        out.close();
    }
    public static void print(Object... values) {for (Object value : values)out.print(value + " ");
        out.println();}
    public static PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
    public static class MyScanner {
        BufferedReader br;
        StringTokenizer st;
        public MyScanner() {
            br = new BufferedReader(new InputStreamReader(System.in));
        }
        String next() {
            while (st == null || !st.hasMoreElements()) {
                try {
                    st = new StringTokenizer(br.readLine());
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            return st.nextToken();
        }
        int nextInt() {return Integer.parseInt(next());}
        long nextLong() {return Long.parseLong(next());}
        double nextDouble() {return Double.parseDouble(next());}
        String nextLine() {
            String str = "";
            try {
                str = br.readLine();
```

```
            } catch (IOException e) {
                e.printStackTrace();
            }
            return str;
        }
    }
}
```

Maths Functions

// **Prime Number**

```
public static boolean isPrime(int n) {
        if (n <= 1)return false;
        if (n == 2 || n == 3)return true;
        if (n % 2 == 0 || n % 3 == 0)return false;
        for (int i = 5; i <= Math.sqrt(n); i += 6) {
            if (n % i == 0 || n % (i + 2) == 0)return false;
        }
        return true;
    }
```

// **Prime Number Up to N**

```
public static List<Integer> getPrime(int n) {
        List<Integer> list = new ArrayList<>();
        boolean[] visi = new boolean[n + 1];
        for (int i = 2; i <= n; i++) {
            if (!visi[i]) {
                list.add(i);
                for (long j = (long) i * i; j <= n; j += i) visi[(int) j] = true;
            }
        }
        return list;
    }
```

// LCM and GCD

```
public static long lcm(long a, long b) {
        return a * b / gcd(a, b);
    }
    public static long gcd(long a, long b) {
        if (a == 0) return b;
        return gcd(b % a, a);
    }
```

// Prime Factors up to MAX

```
static int[]spf;
public static void buildSPF(int MAX) {
        spf=new int[MAX+1];
        for (int i = 2; i <= MAX; i++) spf[i] = i;
        for (int i = 2; i * i <= MAX; i++) {
            if (spf[i] == i) { // i is prime
                for (int j = i * i; j <= MAX; j += i) {
                    if (spf[j] == j) spf[j] = i;
                }
            }
        }
    }
    public static List<Integer> getPrimeFactors(int num) {
        List<Integer> factors = new ArrayList<>();
        while (num > 1) {
            int p = spf[num];
            factors.add(p);
            num /= p;         // Multiple prime number
            // while(num % p == 0) num/=p;  // Unique prime number
        }
        return factors;
    }
```

// Power And Mod

```
public static long powerMod(long base, long exponent, long mod) {
        long result = 1;
        base = base % mod;
        while (exponent > 0) {
            if ((exponent & 1) == 1)result = (result * base) % mod;
            base = (base * base) % mod;
            exponent >>= 1;
        }
        return result;
    }
```

// Check For Sqaure

```
public static boolean isSqr(long val) {
        return ((double)Math.sqrt(val)) == ((long)Math.sqrt(val));
    }
```

// Sum of first k digit from 0 to n

```
static HashMap<Long,Long> map=new HashMap<>();
    public static long sumDigits(long n) {
        if (n < 10) return n * (n + 1) / 2; // base case
        if(map.containsKey(n))return map.get(n);

        long d = (long) Math.log10(n);
        long p = (long) Math.pow(10, d);
        long msd = n / p;

        long ans=msd * sumDigits(p - 1)
                + (msd * (msd - 1) / 2) * p
                + msd * (n % p + 1)
                + sumDigits(n % p);
        map.put(n,ans);
        return ans;
    }
```

// Subtract Mod

```
public static long subMod(long a, long b) {
    a %= MOD; b %= MOD;
    return (a - b + MOD) % MOD;
}
```

// Divisors for N

```
public static List<Integer> getDivisors(int n) {
    List<Integer> divisors = new ArrayList<>();
    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            divisors.add(i);
            if (i != n / i) divisors.add(n / i);
        }
    }
    Collections.sort(divisors);
    return divisors;
}
```

// Combinatorics (nCr % MOD)

```
static long[] fact, invFact;
public static void precomputeFactorials(int n) {
    fact = new long[n + 1];
    invFact = new long[n + 1];
    fact[0] = 1;
```

```
    for (int i = 1; i <= n; i++) fact[i] = (fact[i - 1] * i) % MOD;
    invFact[n] = powerMod(fact[n], MOD - 2, MOD);
    for (int i = n - 1; i >= 0; i--) invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
}
public static long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return (((fact[n] * invFact[r]) % MOD) * invFact[n - r]) % MOD;
}
```

// Extended Euclidean Algorithm

```
public static long[] extendedGCD(long a, long b) {
    if (b == 0) return new long[]{a, 1, 0};
    long[] vals = extendedGCD(b, a % b);
    long g = vals[0], x = vals[2], y = vals[1] - (a / b) * vals[2];
    return new long[]{g, x, y};
}
```

// Modular Inverse (Generic)

```
public static long modInverse(long a, long m) {
    long[] vals = extendedGCD(a, m);
    long g = vals[0], x = vals[1];
    if (g != 1) return -1; // no inverse
    return (x % m + m) % m;
}
```

Data Structure (Object Function)

// ArrayList
```
ArrayList<Integer> list = new ArrayList<>();
list.add(val);              // append
list.add(index, val);       // insert
list.remove(index);         // remove by index
list.get(index);            // access
list.set(index, val);       // update
list.size();                // length
Collections.sort(list);     // sort
Collections.reverse(list); // reverse
```

// Stack
```
Stack<Integer> st = new Stack<>();
st.push(val);
st.pop();
st.peek();
st.isEmpty();
st.size();
```

// Queue
```
Queue<Integer> q = new LinkedList<>();
q.add(val);       // enqueue
q.poll();         // dequeue
q.peek();         // front
q.isEmpty();
```

// Dequeue
```
Deque<Integer> dq = new ArrayDeque<>();
dq.addFirst(val);
dq.addLast(val);
dq.removeFirst();
dq.removeLast();
dq.peekFirst();
dq.peekLast();
```

// PriorityQueue
```
PriorityQueue<Integer> pq = new PriorityQueue<>();              // min-heap
PriorityQueue<Integer> maxpq = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
pq.add(val);
pq.peek();
pq.poll();
```

// TreeMap and TreeSet

```
TreeMap<Integer, Integer> tm = new TreeMap<>();
tm.put(key, value);
tm.higher();
tm.lower();
tm.ceilingKey(x);  // >= x
tm.floorKey(x);    // <= x
tm.remove(key);
TreeSet<Integer> ts = new TreeSet<>();
ts.add(val);
ts.remove(val);
ts.lower(val);
ts.higher(val);
```

// Disjoint Set Union (Union–Find)

```
static int[] parent, size;
static void initDSU(int n) {
    parent = new int[n + 1];
    size = new int[n + 1];
    for (int i = 1; i <= n; i++) { parent[i] = i; size[i] = 1; }
}
static int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}
static void union(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (size[a] < size[b]) { int t = a; a = b; b = t; }
    parent[b] = a;
    size[a] += size[b];
}
```

// Segment Tree

```
public class RSQ {
    private static long[] tree;
    private static int n;

    public RSQ(int[] arr) {
        n = arr.length;
        tree = new long[4 * n];
        build(arr, 1, 0, n - 1);
    }
```

```
private void build(int[] arr, int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(arr, 2 * node, start, mid);
        build(arr, 2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}
public long query(int L, int R) {
    return query(1, 0, n - 1, L, R);
}
private long query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) { // no overlap
        return 0;
    }
    if (L <= start && end <= R) { // total overlap
        return tree[node];
    }
    int mid = (start + end) / 2; // partial overlap
    long leftSum = query(2 * node, start, mid, L, R);
    long rightSum = query(2 * node + 1, mid + 1, end, L, R);
    return leftSum + rightSum;
}
public void update(int idx, int val) {
    update(1, 0, n - 1, idx, val);
}
private void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}
```

Sorting

// HashMap Sorting

```
int[] nums={1,1,1,2,2,3};
    HashMap<Integer, Integer> map = new HashMap<>();
       for (int i = 0; i < nums.length; i++) {
          map.put(nums[i], map.getOrDefault(nums[i], 0) + 1);
       }
       System.out.println(map.entrySet());

       List<Map.Entry<Integer, Integer>> entries = new ArrayList<>(map.entrySet());
       Collections.sort(entries, (entry1, entry2) ->
entry2.getValue().compareTo(entry1.getValue()));
```

// Sorting (Quick Sort)

```
static int partition(int[] arr,int p,int r){
       int pivot=arr[r];
       int i=p-1;
       for(int j=p;j<r;j++){
          if(arr[j]<pivot){
             i++;
             int temp=arr[i];
             arr[i]=arr[j];
             arr[j]=temp;
          }
       }
       System.out.println();
       int temp=arr[i+1];
       arr[i+1]=arr[r];
       arr[r]=temp;
       return i+1;
   }
   static void quickSort(int[] arr,int p,int r){
       if(p<r){
          int q= partition(arr,p,r);
          quickSort(arr, p, q-1);
          quickSort(arr, q+1, r);
       }
   }
```

// 2D Sort

```
Arrays.sort(arr,(a,b)->Integer.compare(a[0],b[0]));
```

Graph Theory

// Dijkstra's algorithm (PQ version)

```
public static int[] dijkstra(int src,List<List<Pair>> graph,int n){
        PriorityQueue<Pair> Q=new PriorityQueue<>(Comparator.comparingInt(pair->pair.cost));
        int dist[] = new int[n];
        Arrays.fill(dist,Integer.MAX_VALUE);
        dist[src]=0;
        Q.add(new Pair(src, 0));
        while(!Q.isEmpty()){
            Pair curr=Q.poll();
            int u = curr.node; // node number
            int cost=curr.cost; // edge codt to node number
            if(cost>dist[u])continue;
            for(Pair neighbor: graph.get(u)){
                int node=neighbor.node;
                int weight=neighbor.cost;
                if(dist[node]>dist[u]+weight){
                    dist[node]=dist[u]+weight;
                    Q.add(new Pair(node,dist[node]));
                }
            }
        }
        return dist;
    }
```

// Topological sort.

```
import java.util.*;
class TopologicalSort {
    // Function to return list containing vertices in Topological order.
    static int[] topoSort(int V, ArrayList<ArrayList<Integer>> adj) {
        int indegree[] = new int[V];
        for (int i = 0; i < V; i++) {
            for (int it : adj.get(i)) {
                indegree[it]++;
            }
        }
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.add(i);
            }
        }
```

```
        int topo[] = new int[V];
        int i = 0;
        while (!q.isEmpty()) {
            int node = q.peek();
            q.remove();
            topo[i++] = node;
            // node is in your topo sort
            // so please remove it from the indegree
            for (int it : adj.get(node)) {
                indegree[it]--;
                if (indegree[it] == 0) {
                    q.add(it);
                }
            }
        }
        return topo;
    }
}
```

// Bellman-Ford algorithm (Shortest path algorithm used to find the minimum distance from a single source vertex to all other vertices in a weighted graph)

```
static class Edge {
        int u, v, w;
        Edge(int u, int v, int w) {
            this.u = u;
            this.v = v;
            this.w = w;
        }
    }
    public static void bellmanFord(int n, List<Edge> edges, int src) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        // Relax all edges (n - 1) times
        for (int i = 0; i < n - 1; i++) {
            for (Edge e : edges) {
                if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.w < dist[e.v]) {
                    dist[e.v] = dist[e.u] + e.w;
                }
            }
        }
        // Check for negative cycle
        for (Edge e : edges) {
            if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.w < dist[e.v]) {
```

```
            System.out.println("Negative weight cycle detected!");
            return;
        }
    }
    System.out.println(Arrays.toString(dist));
  }
```

// FloydWarshall (finds shortest paths between all pairs of vertices in a weighted graph)

```
public static void floydWarshall(int n, int[][] graph) {
    int[][] dist = new int[n][n];
    // Initialize distance matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }
    // Core triple loop
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF &&
                    dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    // Check for negative cycles
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < 0) {
            System.out.println("Negative weight cycle detected!");
            return;
        }
    }
}
```

// Prim's Algorithm (The MST from one vertex by repeatedly choosing the smallest edge connecting the tree to a new vertex)

```
class PrimMST {
   static class Edge {
      int dest, weight;
      Edge(int d, int w) {
         dest = d;
```

```
        weight = w;
    }
  }
  public static void prims(List<Edge>[] graph, int V) {
    boolean[] inMST = new boolean[V];
    int[] key = new int[V];
    int[] parent = new int[V];
    Arrays.fill(key, Integer.MAX_VALUE);
    key[0] = 0;
    parent[0] = -1;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{0, 0}); // (vertex, key)
    while (!pq.isEmpty()) {
      int u = pq.poll()[0];
      inMST[u] = true;
      for (Edge e : graph[u]) {
        int v = e.dest;
        int w = e.weight;
        if (!inMST[v] && w < key[v]) {
          key[v] = w;
          pq.offer(new int[]{v, key[v]});
          parent[v] = u;
        }
      }
    }
    System.out.println("Edges in MST (Prim):");
    for (int i = 1; i < V; i++)
      System.out.println(parent[i] + " - " + i + " : " + key[i]);
  }
}
```

Dynamic Programming (DP)

// Always remember //

IMP :- Always create class array (memo) (ex. Integer[][][] memo)
IMP2 :- For subset problem , Find all posiible sum(can be made) using boolean array for each number (if possible).
1. if array is size of 100 then 2d-dp and for loop allow which leads to n^3(Accepted)
2. if array is size of 1000 then :-
   a.2d-dp without for loop (means 0-1 napsack) (n^2 Accepted)
   b.1d-dp with for loop (means 0-1 napsack) (n^2 Accepted)
   c.2d-dp with for loop (n^3 TLE)
   d.n ≤ 10^6 → Only prefix/suffix tricks or greedy possible.
3. if array is size of greater then 1000 then 1d-dp with for loop (means 0-1 napsack)
4. see 698 for memoization using string and use this when you can't create dp table
5. when you fill memo table, Not always use -1 values , First see Constraints then asign Value.
6. Always check constraints before deciding dimensions (n, sum, etc.)
7. Bitmask DP → for small n (n ≤ 20).

```
// LongestIncreasingSubsequence(NLogN)
import java.util.*;
public class LIS{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.println(lengthOfLIS(arr));
        sc.close();
    }
    private static int lengthOfLIS(int[] nums) {
        ArrayList<Integer> tails = new ArrayList<>();
        for (int x : nums) {
            int pos = Collections.binarySearch(tails, x);
            if (pos < 0) pos = -pos - 1;
            if (pos == tails.size()) tails.add(x);
            else tails.set(pos, x);
        }
        return tails.size();
    }
}
```

// Digit DP

```
// The count of even digits in the number is equal to the count of odd digits and
// number is divisible by k.
public class DigitDP_DigitSumMod3 {
    Integer[][][][][][] memo;
    public int numberOfBeautifulIntegers(int low, int high, int k) {
        int len=String.valueOf(high).length();
        memo=new Integer[len][2][k][len][len][2];
        int ans1=helper(String.valueOf(high),k,0,1,0,0,0,0);
        memo=new Integer[len][2][k][len][len][2];
        int ans2=helper(String.valueOf(low-1),k,0,1,0,0,0,0);
        return ans1-ans2;
    }
    public int helper(String num,int k,int pos,int tight,int mod,int odd,int even,int started){
        if(num.length()==pos){
            return (started==1 && odd==even && mod==0)?1:0;
        }
        if(memo[pos][tight][mod][odd][even][started]!=null)return
memo[pos][tight][mod][odd][even][started];
        int limit = (tight==1)?num.charAt(pos)-'0':9;
        int ans=0;
        for(int i=0;i<=limit;i++){
            int newtight=(tight==1 && i==limit)?1:0;
            int newst = (started == 0 && i == 0) ? 0 : 1;
            int newodd = odd + ((i % 2 == 1 && newst == 1) ? 1 : 0);
            int neweven = even + ((i % 2 == 0 && newst == 1) ? 1 : 0);
            int newmod = newst == 1 ? (mod * 10 + i) % k : mod;
            ans+=helper(num,k,pos+1,newtight,newmod,newodd,neweven,newst);
        }
        return memo[pos][tight][mod][odd][even][started]=ans;
    }
}
/*
We define: dp[pos][tight][mod][odd][even][started]
    Meaning:
        1. pos     → current digit index (0-based)
        2. tight   → 1 if we are restricted by the prefix of the number
        3. mod     → current remainder when the number formed so far is divided by k
        4. odd     → count of odd digits used so far
        5. even    → count of even digits used so far
        6. started → 1 if we have placed a non-zero digit yet (to handle leading zeros)
*/
// pos: index, tight: bound flag, mod: remainder mod k, odd: odd count, even: even count,
started: number started
```

String Algorithms

// All possible Combination

```
static void getCombination(String str,String ans){
        if(str.length()==0){
            System.out.println(ans);
            return;
        }
        getCombination(str.substring(1),ans+str.charAt(0));
        getCombination(str.substring(1), ans);
    }
```

// All possible Permutation

```
static void generatePermutation(String str, int start, int end) {
        if (start == end - 1) {
            System.out.println(str);
        } else {
            for (int i = start; i < end; i++) {
                str = swapString(str, start, i);
                generatePermutation(str, start + 1, end);
                str = swapString(str, start, i);
            }
        }
    }
    public static String swapString(String a, int i, int j) {
        char[] b = a.toCharArray();
        char ch = b[i];
        b[i] = b[j];
        b[j] = ch;
        return String.valueOf(b);
    }
```

// Z-Algorithum

```
/* Z-array Z[i] = length of the longest substring starting at i that matches the prefix of string. */
public class ZAlgorithm {
    // Compute Z-array
    public static int[] computeZArray(String s) {
        int n = s.length();
        int[] Z = new int[n];
        int L = 0, R = 0;
        for (int i = 1; i < n; i++) {
            if (i <= R) {
```

```
            Z[i] = Math.min(R - i + 1, Z[i - L]);
        }
        while (i + Z[i] < n && s.charAt(Z[i]) == s.charAt(i + Z[i])) {
            Z[i]++;
        }
        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
    return Z;
}
// Pattern search using Z (Matching)
public static List<Integer> searchPattern(String text, String pattern) {
    String combined = pattern + "$" + text;
    int[] Z = computeZArray(combined);
    List<Integer> result = new ArrayList<>();
    int patternLength = pattern.length();
    for (int i = 0; i < Z.length; i++) {
        if (Z[i] == patternLength) {
            result.add(i - patternLength - 1); // index in text
        }
    }
    return result;
}
public static void main(String[] args) {
    String text = "aabxaayaab";
    String pattern = "aab";
    List<Integer> matches = searchPattern(text,pattern);
    System.out.println("Pattern found at indices: " + matches);
}
}
```

// Manacher's Algorithm

```
/* Find the longest palindromic substring in O(n) time */
public class Manacher {
    public static String longestPalindrome(String s) {
        // Preprocess
        StringBuilder sb = new StringBuilder("^");
        for (char c : s.toCharArray()) sb.append("#").append(c);
        sb.append("#$");
        String t = sb.toString();
        int n = t.length();
        int[] P = new int[n];
```

```
        int C = 0, R = 0;
        for (int i = 1; i < n - 1; i++) {
            int mirr = 2 * C - i; // mirror of i around C
            if (i < R) P[i] = Math.min(R - i, P[mirr]);
            // Expand palindrome centered at i
            while (t.charAt(i + (1 + P[i])) == t.charAt(i - (1 + P[i]))) P[i]++;
            // Update C and R
            if (i + P[i] > R) {
                C = i;
                R = i + P[i];
            }
        }
        // Find max palindrome
        int maxLen = 0, centerIndex = 0;
        for (int i = 1; i < n - 1; i++) {
            if (P[i] > maxLen) {
                maxLen = P[i];
                centerIndex = i;
            }
        }
        int start = (centerIndex - maxLen) / 2;
        return s.substring(start, start + maxLen);
    }
    public static void main(String[] args) {
        String s = "babad";
        System.out.println(longestPalindrome(s)); // Output: "bab" or "aba"
    }
}
```

RandomTestGenerator

```
static Random rand = new Random();
    /** Generates random integer in [l, r] */
    public static int randInt(int l, int r) {
        return l + rand.nextInt(r - l + 1);
    }
    /** Generates random array of size n, values in [minVal, maxVal] */
    public static int[] randomArray(int n, int minVal, int maxVal) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = randInt(minVal, maxVal);
        return arr;
    }
    /** Generates random 2D matrix */
    public static int[][] randomMatrix(int n, int m, int minVal, int maxVal) {
        int[][] mat = new int[n][m];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                mat[i][j] = randInt(minVal, maxVal);
        return mat;
    }
    /** Generates random string of lowercase letters */
    public static String randomString(int len) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < len; i++)
            sb.append((char) ('a' + rand.nextInt(26)));
        return sb.toString();
    }
    /** Generates random edges for graph (can be weighted/unweighted) */
    public static List<int[]> randomGraph(int n, int m, boolean weighted) {
        List<int[]> edges = new ArrayList<>();
        Set<String> used = new HashSet<>();
        while (edges.size() < m) {
            int u = randInt(0, n - 1);
            int v = randInt(0, n - 1);
            if (u == v || used.contains(u + "," + v)) continue;
            used.add(u + "," + v);
            if (weighted)  edges.add(new int[]{u, v, randInt(1, 20)});
            else  edges.add(new int[]{u, v});
        }
        return edges;
    }
```

Bit_Manipulation Tricks

```
// 1. Check if a number is Even or Odd
public boolean isEven(int x){
    return (x & 1) == 0;
}
// 2. Check where ith bit is set
public boolean isSet(int n,int i){
    return (n & (1 << i)) != 0;
}
// 3. Set ith bit
public int set(int n,int i){
    return n | (1 << i);
}
// 4. Unset ith bit
public int unSet(int n,int i){
    return n & (~(1 << i));
}
// 5. Set ith bit
public int togglebit(int n,int i){
    return n ^ (1 << i);
}
// 6. is power of 2
public boolean isTwoPow(int n){
    return (n>0) && ((n & (n-1)) == 0);
}
// 7. Brian Kernighan's Algorithm
public int countSetBit(int n){
    int count = 0;
    while (n > 0) {
        n = n & (n - 1); // This clears the least significant bit
        count++;
    }
    return count;
  }
// 9. Get MSB index
public int getMSB(int n) {
    return 31-Integer.numberOfLeadingZeros(n);
}
// 10. Get LSB index
public int getLSB(int n) {
    return Integer.numberOfTrailingZeros(n);
}
```

```
    // 11. Differnt type of bit set
    If you want the actual bit value (0/1) → use (num >> i) & 1.
    (Best for trie navigation where you need index 0 or 1.)
    If you just want to test if the bit is set → use num & (1 << i) and check if it's != 0.
    (Best for boolean conditions.)

    // 12. AND trick
    let a b are 2 numbers
    if a&b == 0 then a^b = a+b
}
```

Trie

```
public static class TrieNodeW{
        TrieNodeW[] next=new TrieNodeW[26];
        String word;
    }
    public static TrieNodeW build(String[] arr){
        TrieNodeW root=new TrieNodeW();
        for(String str:arr){
            TrieNodeW curr=root;
            for(char ch:str.toCharArray()){
                if(curr.next[ch-'a']==null)curr.next[ch-'a']=new TrieNodeW();
                curr=curr.next[ch-'a'];
            }
            curr.word=str;
        }
        return root;
    }
```

Pattern

Pattern 1: Greedy Algorithms
The most common pattern. You must learn to trust the simplest choice. The core idea is to make the locally optimal choice at each step, hoping it leads to a global optimum. Your intuition must tell you when this works.

Pattern 2: Binary Search on the Answer
This pattern is a competitive programming superpower. If a problem asks you to find the "minimum possible maximum value" or "maximum possible minimum value," your brain should scream "BINARY SEARCH!"

Pattern 3: Constructive Algorithms & Number Theory
These problems ask you to build a solution that meets certain criteria. Often, the solution relies on a simple mathematical property (like parity - even/odd) or a clever observation.

Pattern 4: Two Pointers / Sliding Window
This is an essential technique for array/string problems to avoid O(N^2) complexity. It involves maintaining two pointers (left and right) to define a "window" or subarray and moving them intelligently.
Note :- If to be find answer in given range(like lower and upper) then find value for 0 to upper and value for 0 to lower , like ans = upper(value)-lower(value)
// Easy To find ,But not always optimised

Pattern 5: Number Theory / Math
This pattern covers specialized mathematical algorithms for handling operations on large integers, particularly when dealing with primes, modulo, and combinatorics.

Pattern 6: Disjoint Set Union (DSU) / Union-Find
This is your tool for **grouping** and **connecting**. If a problem involves merging components, checking for connectivity, or finding "families" of items, DSU is the answer. Its power is doing this in *near-constant time* (using optimizations like path compression and union by size/rank).

Pattern 7: HashMap / HashSet
When you need to "remember" or "count" things instantly by a key. If the problem involves **counting frequencies** of numbers, strings, or objects, or "mapping" one value to another (like a name to a number), your brain should scream "HASH MAP!"

Pattern 8: Graph Traversal (General)
If you can model your problem as a set of **states** (nodes) and **transitions** between states (edges), you can often solve it with a simple graph traversal.

Pattern 9: Dynamic Programming (DP)

DP is about solving a big problem by breaking it into smaller, **overlapping** sub-problems. The key is to ask, "Have I solved this exact sub-problem before?" If so, "remember" the answer (memoization) and reuse it.

Pattern 10: Bit Manipulation & Bitmasks

When the constraints are tiny (e.g., n <= 20), it's a huge clue. You can represent **subsets** or **states** as a single integer (a bitmask). Each bit (0 or 1) represents the presence or absence of an element. This is often used for "DP on Subsets" or "iterating through all possibilities."

Pattern 11: Trie (Prefix Tree)

When your problem involves a dictionary of strings, string-matching, or **prefixes**, a Trie is your go-to. It stores strings in a tree where each node is a character and a path from the root is a prefix. Its magic is finding all strings with a common prefix in $O(L)$ time (where L is prefix length).

Note: A Trie's main advantage over a set<String> is that it stores *shared prefixes* only once, making it much more efficient for large sets of strings with common starting parts.

Pattern 12: Stack

Think "Last-In, First-Out." Stacks are perfect for "matching" items (like parentheses), "undoing" actions, or any problem where you need to process things in reverse order. They are also the key to the **"Next Greater/Smaller Element"** pattern

Pattern 13: Priority Queue (Heap)

When you need to process items "in order," but you are also adding new items dynamically. It's the "get me the **best** (max/min) item so far" data structure. Use it when you need to maintain a "top K" list, or for greedy algorithms where you always "process the best available item."

Pattern 14: BFS / DFS (Traversal Basics)

BFS (Breadth-First Search): Explores *layer-by-layer* using a **Queue**. It is your go-to for shortest paths in unweighted graphs.

DFS (Depth-First Search): Explores *as deep as possible* using **Recursion** (or an explicit stack) before backtracking. It is great for connectivity, cycle detection, and topological sorting.

Note on DFS Stack Depth at max 20000.

Pattern 15: Segment Tree

This is the ultimate tool for efficiently answering **range queries** (e.g., *range sum*, *range minimum*) and **range updates** on an array in O(log N) time.

If a problem requires you to calculate a property (like sum, min, max, or GCD) over a **contiguous sub-array range** and also involves **modifying specific elements or ranges** of the array, your brain should scream "SEGMENT TREE!".